



µLab 2 • Week 2 / Day 2 • Driver's handout

Measuring the speed-up from one worker to four

Group _____ Date _____ Time _____

Driver (types) _____ Mapper (looks up) _____ Referee (demos) _____

STEP 1 • TAKE THE ONE-WORKER CLOCK AGAIN

What this round is meant to show, before you start. Four workers have four times the cores of one. If the work divides cleanly, the same job should finish in a quarter of the time. It will not, and the size of the gap is the measurement.

Start from one worker, so the two clocks compare. If µLab 1's number is still on your sheet and nobody has touched the cluster since, copy it and go to step 2.

All of this round happens in the **kubectl client**, the one µLab 1 sent you to look up. If you closed it, open it again the same way.

IN THE KUBECTL CLIENT

```
kubectl scale deployment spark-worker --replicas=1
kubectl get pods
```

`scale ... --replicas=1` States how many workers you want. It returns immediately; the cluster then removes pods until it agrees. `get pods` is how you find out that it has.

The master publishes a web page of its own, the **Spark master UI**, listing the workers that have registered and the cores they bring. Start it once and leave it running.

ON YOUR LAPTOP

```
docker run -d --name ui -p 8080:8080 -v "$PWD/kube:/kube:ro" \
-e KUBECONFIG=/kube/config alpine/kubectl \
--server https://host.docker.internal:6443 --insecure-skip-tls-verify \
port-forward --address 0.0.0.0 deploy/spark-master 8080:8080
```

`-p 8080:8080` Publishes the UI on your laptop. This is the argument that makes it reachable from a browser. From inside a container, your laptop is called `host.docker.internal`, not `127.0.0.1`.
`--server ... host.docker.internal` Tunnels the UI out through the API server, the one port the cluster already publishes. Nothing about the cluster changes.

Open `http://127.0.0.1:8080/`, with the `http://` typed out. The summary at the top lists **Alive Workers** on one line and **Cores in use** on the next; every number this round asks for is read from those two lines. Wait until **Alive Workers** reads 1 before you start the job.

Page does not open? Panel 7. Remove it at the end of the round with `docker rm -f ui`.

```
kubectl exec deploy/spark-master -- \
/opt/spark/bin/spark-submit --master spark://spark-master:7077 \
/lab/wordcount_df.py /lab/data /lab/data/simplewiki.txt.bz2
```

Submit it **twice**, and write down the second number. A first run pays to pull the file off the disk; a second finds it already in memory, and that gap has nothing to do with workers. Step 2 asks for a second run too, so the two numbers you compare are measured the same way.

elapsed A, second run, one worker _____ input partitions _____

STEP 2 • GROW THE CLUSTER TO FOUR, AND RUN THE IDENTICAL JOB

Before you touch anything, read **Cores in use** off the Spark master UI. It is written `N Total, M Used`; write down the **Total**. You cannot get this number back once the cluster has grown.

Cores in use, one worker _____

IN THE KUBECTL CLIENT

```
kubectl scale deployment spark-worker --replicas=4
kubectl get pods
```

Now reload the Spark master UI. **Alive Workers** and **Cores in use** both climb as the new workers register. Once **Alive Workers** reads 4, take the second reading and run the same submission as step 1, unchanged. Twice again, and write down the second number, for the reason step 1 gave.

Cores in use, four workers _____

elapsed B, second run, four workers _____ input partitions _____

Spark cut the same input into 2 chunks the first time and 4 the second. The number of chunks follows the number of cores it has to keep busy, not the size of what it is reading.

STEP 3 · WRITE THE SPEED-UP, AND SAY WHY IT IS NOT FOUR

Four workers have four times the cores of one. Divide A by B, and see how much of that four you actually got.

A ÷ B _____

Whatever you wrote, it is not four. Part of a Spark job does not divide at all: starting it costs the same on one worker as on forty, and no number of workers makes that part shorter.

Write one sentence: what does that mean for a team whose answer to a slow job is always to add machines?

Solved means panels 1 to 3 are done: two timings, the speed-up, and one sentence on the part that does not divide. The referee ticks the bar below at that moment, not at the caucus.

Reached this line? Then you have solved the challenge.



Referee: tick the box on the left now, and write the time. Do not wait for the caucus. Then carry on below — the extensions are not optional, they are the rest of the round.

Solved at _____ Referee's initials _____

4 · THE NUMBERS THIS WAS MEASURED AGAINST

Observed on 2026-09-23 on a ten-core laptop, against `simplewiki.txt.bz2` (212 354 articles, 259 MB of text), each worker configured with one core:

input	workers	partitions	elapsed
.bz2	1	2	27.9 s
.bz2	4	4	14.1 s
plain .txt	1	3	11.6 s
plain .txt	4	4	8.9 s

Four times the workers bought less than a factor of **two**, and your own run may have bought less than that again. It is not the cost of starting a job: the same script on a single line of text finishes in about two seconds whether one worker is running or four. So whatever refuses to divide here is work that grows with the data, and panel 6 sends you to find where it puts the ceiling.

Read the last two rows before you do panel 5. They say that the **file format** was worth more than the extra workers.

Your numbers will be slower than these and will differ a lot from the laptop next to you: seventeen clusters on one wifi, and whatever else each machine is running. What survives that is the ratio between two of your own rows, not any single number.

5 • EXTENSION • TAKE THE COMPRESSION AWAY

What this shows. A compressed stream has to be decoded before it can be split, and that decode is serial in long runs. Give Spark the same text with nothing to decode and the ceiling moves.

ON YOUR LAPTOP

```
bzcat lab/data/simplewiki.txt.bz2 > lab/data/simplewiki.txt
ls -lh lab/data/simplewiki.txt
```

It becomes 260 MB on disk. Delete it at the end of the round if you are short of space.

Run the same submission with the new path, at four workers and then at one.

plain text, four workers _____ plain text, one worker _____

Now put the four clocks in one place and answer: which bought more, four times the workers or dropping the compression? And what did dropping the compression cost, in disk and in download?

6 • EXTRA MILE • FIND WHERE THE CEILING ACTUALLY IS

Scale to eight workers and run the plain-text job again. The laptop has ten cores, so eight workers of one core each still fit.

eight workers, plain text: _____

Compare with the four-worker number and say what is now the limit. The answer is not “the laptop”: the plan tells you how many pieces the input came in, and no worker can be given a ninth piece that does not exist.

7 • ANNEX • THREE THINGS THAT GO WRONG, AND HOW TO FIX THEM

Why steps 1 and 2 ask for the second run	The operating system keeps recently read files in memory. The first run of a file pays for the disk, every run after it does not, and the difference is large enough to swamp what you are trying to measure. Comparing a first run against a second would tell you about the disk instead of about the workers.
The page will not open	Check three things, in order. One , <code>docker ps</code> should list <code>ui</code> ; if it exited, <code>docker logs ui</code> says why. Two , the address must begin <code>http://</code> : the UI is served over plain HTTP and Firefox will silently try <code>https</code> otherwise, which fails with no useful message. Three , if you ran <code>kubectctl port-forward</code> yourself instead of the container above, it must not be from inside the <code>kubectctl</code> client: that container is on the cluster's network rather than your laptop's, so it forwards to a <code>localhost</code> your browser cannot reach.
A job that waits	If a previous submission is still registered, yours gets no cores and simply waits, reporting a clock that is mostly queueing. In the client, <code>kubectctl logs deploy/spark-master grep "Registered app"</code> shows what the master thinks is running; <code>kubectctl delete pod -l app=spark-master</code> clears a stuck one.